

Практическая работа 34. Создание приложения для поиска рейсов

Необходимые условия

- Базовые знания SQL для чтения и работы с реляционной базой данных.
- Умение использовать Room в приложении для Android для чтения из базы данных и записи в нее.
- Умение использовать DataStore для хранения простых данных.
- Умение создавать умеренно сложные пользовательские интерфейсы с помощью Compose.

Что вы будете создавать

- Приложение для Android, которое запрашивает у пользователя аэропорт вылета, выполняет поиск по заранее заполненной базе данных, чтобы представить список рейсов, вылетающих из этого аэропорта, позволяет пользователю сохранять избранные рейсы и обновляет базу данных с помощью Room.

Что вам понадобится

- Компьютер с Android Studio

Обзор

Поздравляем вас с завершением изучения Главы 6! Вы начали с общего обзора реляционных баз данных и языка структурированных запросов (SQL), интегрировали реляционную базу данных в приложение с помощью Room и узнали о Preferences DataStore для сохранения настроек и состояния пользовательского интерфейса. Пришло время применить полученные знания на практике.

В этом проекте вы создадите приложение для поиска рейсов, в котором пользователи вводят аэропорт и могут просмотреть список направлений, использующих этот аэропорт в качестве пункта отправления. Этот проект дает вам возможность отработать навыки работы с SQL, Room и DataStore, предлагая вам набор требований к приложению, которые вы должны выполнить. В частности, приложение «Поиск рейсов» должно отвечать следующим требованиям:

Предоставить текстовое поле для ввода пользователем названия аэропорта или идентификатора аэропорта Международной ассоциации воздушного транспорта (IATA). Запрос к базе данных для предоставления предложений с автозаполнением по мере ввода пользователем. Когда пользователь выбирает предложение, сформируйте список доступных рейсов из этого аэропорта, включая идентификатор IATA и название аэропорта, в другие аэропорты в базе данных. Позволяет пользователю сохранять избранные индивидуальные маршруты. Если поисковый запрос не введен, отобразите все выбранные пользователем любимые маршруты в виде списка. Сохраняйте текст поискового запроса в Preferences DataStore. Когда пользователь снова откроет приложение, текст поиска, если таковой имеется, должен быть предварительно заполнен текстовым полем с соответствующими результатами из базы данных. Для этого проекта мы предоставили уже заполненную базу данных. Однако предполагается, что вы создадите приложение с нуля в соответствии с требованиями - практика для реальной работы в качестве разработчика Android. Этот проект также является хорошим шансом пересмотреть или отточить свои навыки создания пользовательского интерфейса с помощью Compose,

поскольку вам не нужно было делать много работы с пользовательским интерфейсом со времен Главы 4.

Получение базы данных рейсов

Данные для этого приложения поступают из базы данных рейсов. База данных рейсов содержит две таблицы, аэропорт и фаворит.

Таблица аэропорта содержит следующую схему.

Column	Data type	Description
id	INTEGER	Уникальный идентификатор (первичный ключ)
iata_code	VARCHAR	3-буквенный код IATA
name	VARCHAR	Полное название аэропорта
passengers	INTEGER	Количество пассажиров в год

Таблица favorite содержит следующую схему.

Column	Data type	Description
id	INTEGER	Уникальный идентификатор (первичный ключ)
departure_code	VARCHAR	IATA-код отправления
destination_code	VARCHAR	IATA-код пункта назначения

Вы можете использовать таблицу аэропортов для поиска аэропортов и создания списка потенциальных рейсов. Таблица favorite, которая изначально пуста, используется для сохранения пар направлений вылета и прилета, выбранных пользователем.

Загрузите файл `flight_search.db` из ветки проекта репозитория SQL Basics GitHub [здесь](#).

Спланируйте свое приложение

Спланируйте свой пользовательский интерфейс. Вы можете разрабатывать свое приложение так, как вам нравится. В качестве руководства нижеприведенные описания и изображения являются примером того, что пользователь может ожидать увидеть в приложении.

Когда пользователь впервые открывает приложение, он видит пустой экран с текстовым полем, предлагающим ввести аэропорт.

Когда пользователь начинает вводить текст, приложение отображает список предложений с автозаполнением, которые соответствуют названию или идентификатору аэропорта.



Flight Search

🔍 Int



SVO Sheremetyevo - A.S. Pushkin international airport

MUC Munich International Airport

DUS Düsseldorf International Airport

ATH Athens International Airport

LYS Lyon-Saint Exupéry Airport

FCO Leonardo da Vinci International Airport

VIE Vienna International Airport

KEF Keflavík International Airport



Int

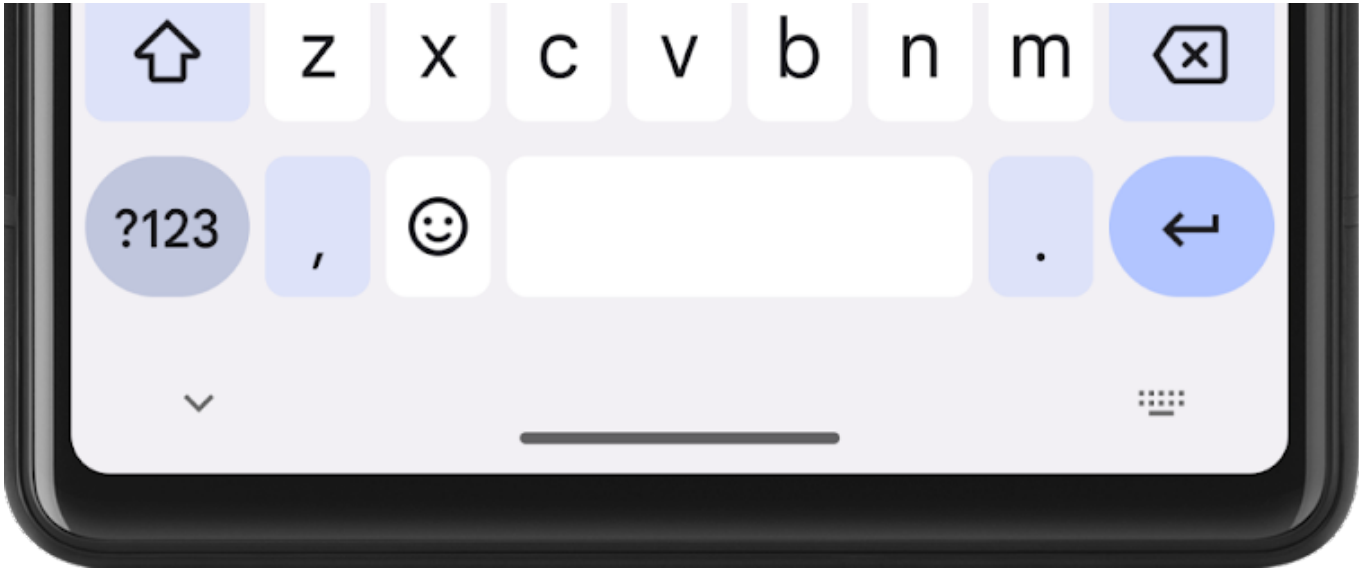
Interesting

Into



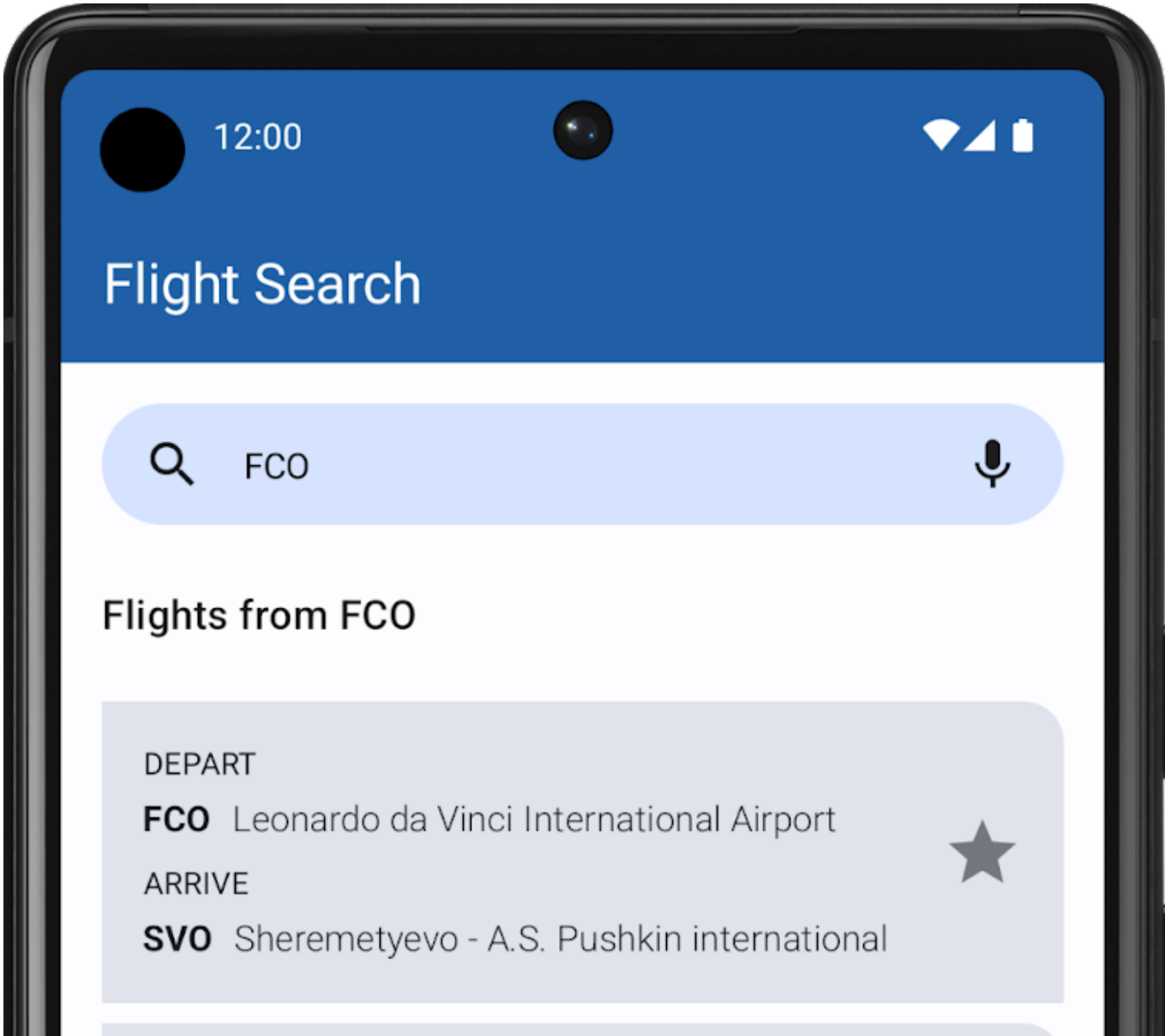
q¹ w² e³ r⁴ t⁵ y⁶ u⁷ i⁸ o⁹ p⁰

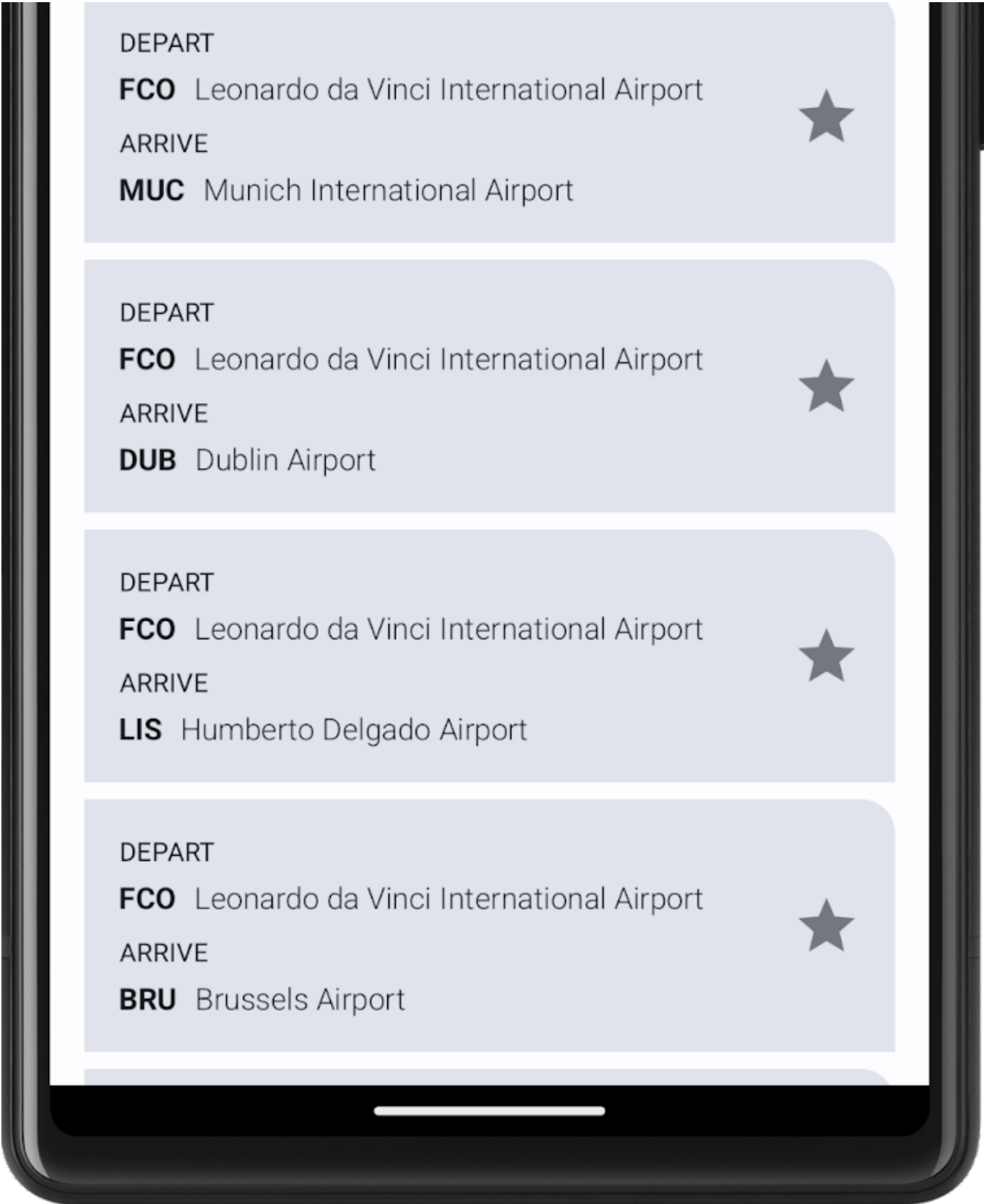
a s d f g h j k l



{style=«width:400px»}

Когда пользователь выбирает предложение, приложение отображает список всех возможных рейсов из этого аэропорта. Каждый пункт включает в себя идентификатор и названия обоих аэропортов, а также кнопку для сохранения пункта назначения в качестве избранного. Не стесняйтесь экспериментировать с макетом, лишь бы он передавал всю необходимую информацию.





{style="width:400px"}

When the user clears the search box or does not enter a search query, the app displays a list of saved favorite destinations, if any exist.



Flight Search

 Enter departure airport



Favorite routes

DEPART

FCO Leonardo da Vinci International Airport

ARRIVE

CPH Copenhagen Airport



DEPART

OSL Oslo Airport

ARRIVE

LIS Humberto Delgado Airport





{style="width:400px"}

Tip: Use a LazyColumn to display autocomplete suggestions and search results. You might want to wrap your layout in a Box and use animation APIs to display the autocomplete suggestions in front of the search results list. Your UI then has two lazy columns: the search results, which the app always displays, and the autocomplete suggestions, which the app displays conditionally while the user types.

Use Room to integrate the flights database. In order to implement the features above, you need to leverage your knowledge of SQL and Room. The database already consists of two tables, airport and favorite, and you need entities for each one. Select the appropriate Kotlin data types so that you can access the values in each table.

Additionally, you need to consider the following requirements when querying the flights database and persisting data:

Search for autocomplete suggestions in the airport table. Keep in mind that the user might already know the airport code, so you need to check their input against the `iata_code` column, in addition to the name column, when searching for text. Remember that you can use the LIKE keyword to perform text searches. Show more frequently visited airports in descending order by sorting on the passengers column. Assume that every airport has flights to every other airport in the database (except for itself). When no text is in the search box, display a list of favorite flights, showing the departure and destination. As the favorite table only includes columns for the airport codes, you're not expected to show the airport names in this list. Perform all database querying with SQL and Room APIs. The whole point is to NOT load your entire database into memory at once, only to retrieve the required data as needed. Persist user state with Preferences DataStore. In addition to SQL and Room, you also know how to persist individual values like user settings. For the Flight Search app, you need to store the user's search string in Preferences DataStore so that it populates when the user relaunches the app. If the text field is empty when the user exits the app, then the list of favorite flights needs to display instead.

5. Build the Flight Search app

Now that you've read through all the requirements, it's time to build your app. Although this unit focuses exclusively on data persistence, it's important to continue to get cumulative practice. While you've seen

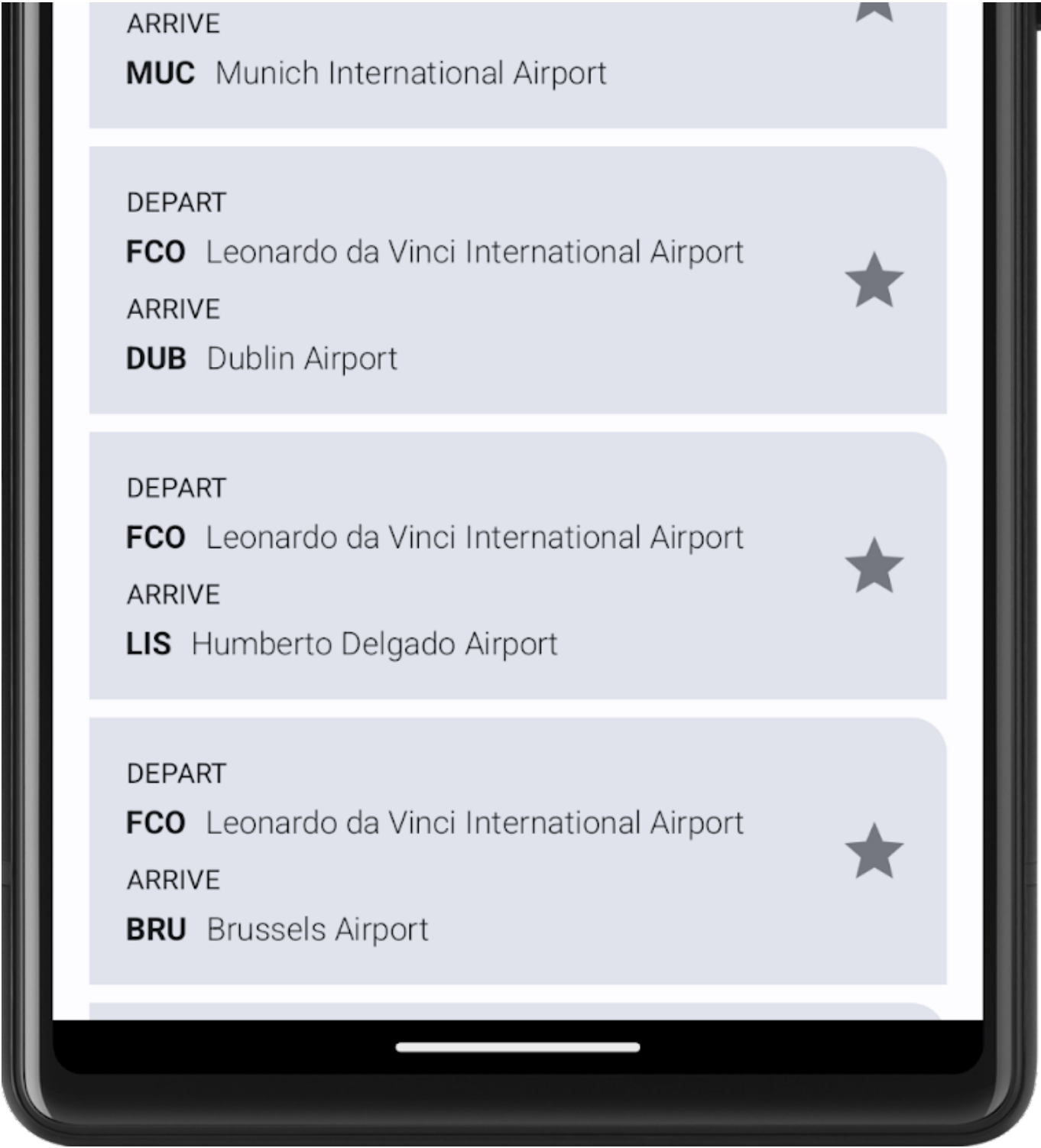
example screenshots of the Flight Search app in action, this project is your opportunity to make the app your own and stand out.

Although these exact tasks are unfamiliar, you already know all the core concepts necessary to build this project. If you get stuck or need a refresher, you can refer to the previous codelabs.

The following might be useful to you as you work on your project:

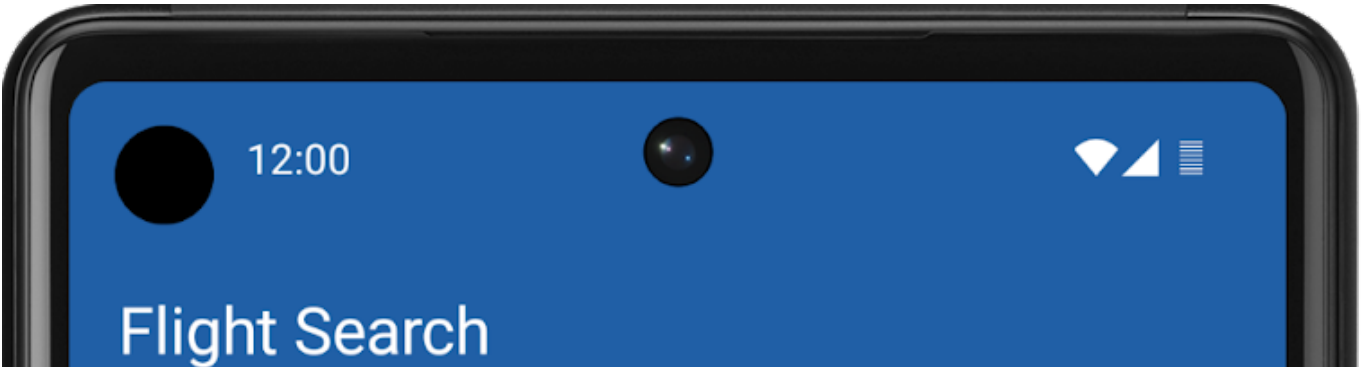
Use SQL to read and write to a database Persist data with Room Read and update data with Room Save preferences locally with DataStore Most importantly, enjoy the process! Learning is a journey. Even if you find this project challenging, you'll probably learn something new and then find the same problems easy to solve the next time. Have fun, and see you in the next unit!Когда пользователь выбирает предложение, приложение отображает список всех возможных рейсов из этого аэропорта. Каждый пункт включает в себя идентификатор и названия обоих аэропортов, а также кнопку для сохранения пункта назначения в качестве избранного. Не стесняйтесь экспериментировать с макетом, лишь бы он передавал всю необходимую информацию.





{style=«width:400px»}

Когда пользователь очищает поле поиска или не вводит поисковый запрос, приложение отображает список сохраненных любимых направлений, если таковые существуют.



 Enter departure airport



Favorite routes

DEPART

FCO Leonardo da Vinci International Airport

ARRIVE

CPH Copenhagen Airport



DEPART

OSL Oslo Airport

ARRIVE

LIS Humberto Delgado Airport





{style=«width:400px»}

Совет: Используйте LazyColumn для отображения предложений автозаполнения и результатов поиска. Возможно, вы захотите обернуть макет в Box и использовать API анимации для отображения предложений автозаполнения перед списком результатов поиска. Тогда в вашем пользовательском интерфейсе будет две «ленивые» колонки: результаты поиска, которые приложение отображает всегда, и предложения автозаполнения, которые приложение отображает условно, пока пользователь набирает текст.

Использование Room для интеграции базы данных рейсов Чтобы реализовать описанные выше функции, вам необходимо использовать знания SQL и Room. База данных уже состоит из двух таблиц - airport и favorite, и вам нужны сущности для каждой из них. Выберите подходящие типы данных Kotlin, чтобы можно было получить доступ к значениям в каждой таблице.

Кроме того, при запросе к базе данных рейсов и сохранении данных необходимо учитывать следующие требования:

Поиск предложений с автозаполнением в таблице аэропортов. Помните, что пользователь может уже знать код аэропорта, поэтому при поиске текста необходимо проверять вводимые им данные по столбцу iata_code в дополнение к столбцу name. Помните, что для выполнения текстового поиска можно использовать ключевое слово LIKE. Покажите более часто посещаемые аэропорты в порядке убывания, отсортировав их по столбцу «Пассажиры». Предположим, что каждый аэропорт выполняет рейсы во все остальные аэропорты в базе данных (за исключением самого себя). Если в поле поиска нет текста, выведите список избранных рейсов с указанием пункта отправления и назначения. Поскольку таблица избранных рейсов содержит только столбцы для кодов аэропортов, отображение названий аэропортов в этом списке не предполагается. Все запросы к базе данных выполняйте с помощью SQL и API Room. Суть в том, чтобы НЕ загружать всю базу данных в память сразу, а извлекать нужные данные по мере необходимости. Сохраняйте состояние пользователя с помощью Preferences DataStore Помимо SQL и Room, вы также знаете, как сохранять отдельные значения, например настройки пользователя. Для приложения Flight Search вам нужно сохранить строку поиска пользователя в Preferences DataStore, чтобы она заполнялась при повторном запуске приложения. Если при выходе из приложения текстовое поле будет пустым, то вместо него должен отображаться список избранных рейсов.

Создайте приложение для поиска рейсов

Теперь, когда вы ознакомились со всеми требованиями, пришло время создать приложение. Хотя этот блок посвящен исключительно сохранению данных, важно продолжать получать накопительную

практику. Хотя вы уже видели примеры скриншотов приложения Flight Search в действии, этот проект - ваша возможность сделать приложение своим и выделить его на фоне других.

Хотя эти задачи вам незнакомы, вы уже знаете все основные понятия, необходимые для создания этого проекта. Если вы застряли или нуждаетесь в обновлении, вы можете обратиться к предыдущим кодовым лекциям.

В процессе работы над проектом вам может пригодиться следующее:

- Использование SQL для чтения и записи в базу данных
- Сохранение данных с помощью Room
- Чтение и обновление данных с помощью Room
- Сохраняйте предпочтения локально с помощью DataStore

Самое главное - наслаждайтесь процессом! Обучение - это путешествие. Даже если этот проект покажется вам сложным, вы наверняка узнаете что-то новое и в следующий раз легко решите те же проблемы. Веселитесь, и до встречи в следующей части!